



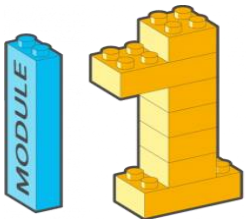
VIT[®]

Vellore Institute of Technology
(Deemed to be University under section 3 of UGC Act, 1956)

BCSE307L_PRINCIPLES OF COMPILER DESIGN



INTRODUCTION TO COMPILATION AND LEXICAL ANALYSIS



Dr. B.V. Baiju,
SCOPE, Assistant Professor
VIT, Vellore

LEXEME

- The sequence of characters matched by a pattern to form the corresponding token or a sequence of input characters that comprises a single token is called a lexeme
- Words in the source program.
- In preprocessing Comment, Pre-processor directive, Macro, Whitespace are removed
- Example: Given the expression·

c = a + b * 5

LEXEMES	TOKENS
c	identifier
=	assignment symbol
a	identifier
+	+(addition symbol)
b	identifier
*	*(multiplication symbol)
5	5(number)

w h i l e (1 3 7 < i) \n \t + + i ;

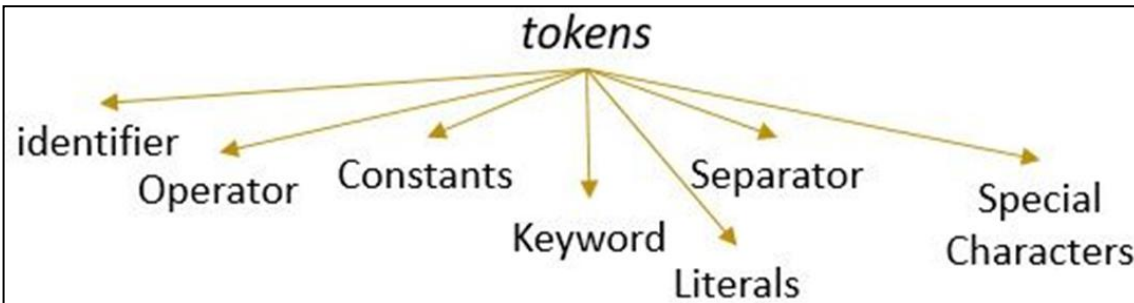
The piece of the original program from which we made the token is called a **lexeme**.

T_While

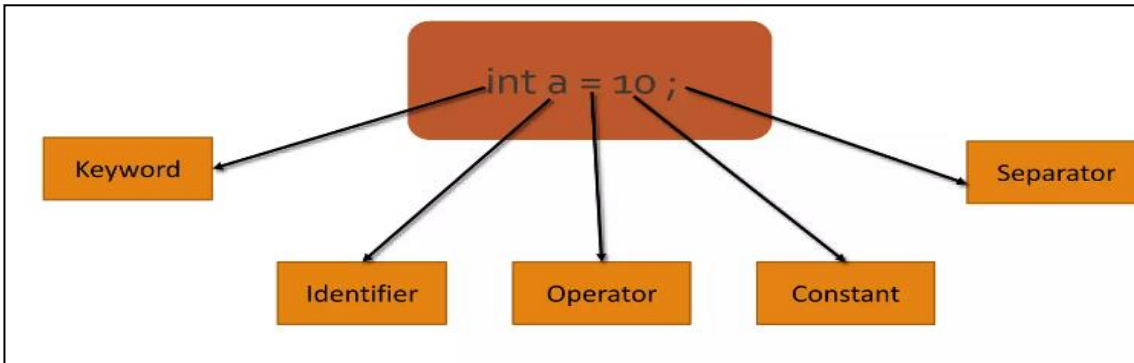
This is called a **token**. You can think of it as an enumerated type representing what logical entity we read out of the source code.

TOKENS

- Token is a **sequence of characters in the input** that form a meaningful word.
- A *token* is a pair consisting of a **token name** and an optional **attribute value**.
- The token names are the input symbols that the parser processes
- In most languages, the tokens fall into these categories:



Keywords	for, while, if etc
Identifier	Variable name, function name
Operators	'+', '++', '-' etc.
Separators	','';'



Example of Non-Tokens:

- Comments
- Preprocessor directive
- Macros
- Blanks
- Tabs
- Newline

Lexeme	Token	Description
if, else, for	Keyword	Reserved words
score, s2, test	Identifier	Any combination of letters and digits
2.765, 0, 4.03e2, 10	Number	Numbers and Constants
+, -, <, >, >=, <=, !=	Operators	Operators
“Welcome back”	Literal	Anything within Quotes
(, {, , , ;, :, ',	Punctuation Symbol	Parenthesis and Separators

```

int main()
{
    int x,a=2,b=3,c=5;
    x = a+b*c;
    printf("The value of x is %d",x);

    return 0;
}

```

Count: 39

Example : Consider the following code that is fed to Lexical Analyzer

```
#include <stdio.h>
int maximum(int x, int y)
{
    // This will compare 2 numbers
    if (x > y)
        return x;
    else {
        return y;
    }
}
```

Non-Tokens

Type	Examples
Comment	// This will compare 2 numbers
Pre-processor directive	#include <stdio.h>
Pre-processor directive	#define NUMS 8,9
Macro	NUMS
Whitespace	/n /b /t

Tokens

Lexeme	Token
int	Keyword
maximum	Identifier
(	Operator
int	Keyword
x	Identifier
,	Operator
int	Keyword
y	Identifier
)	Operator
{	Operator
if	Keyword

- **Example :** Find out the Lexemes and Tokens for the given code

```
#include<iostream>
// example
int main()
{
    int a, b;
    a = 7;
    return 0;
}
```

TOKENS

LEXEMES	TOKENS
int	Keyword
main	Keyword
(	Operator
)	Operator
{	Operator
int	Keyword
a	Identifier
,	Separator
b	Identifier
;	Operator
a	Identifier
=	assignment symbol
10	10(number)
;	Operator
return	Keyword
0	0(number)
;	Operator
}	operator

NON TOKENS

TYPE	TOKENS
preprocessor directive	#include<iostream>
comment	// example

Example

1. For the given program find out the valid tokens

```
int main()
{
    // 2 variables
    int a, b;
    a = 10;
    return 0;
}
```

Valid tokens

```
'int' 'main' '(' ')' '{' 'int' 'a' ',' 'b' ';'
'a' '=' '10' ';' 'return' '0' ';' '}'
```

2. Count number of tokens

```
int main()
{
    int a = 10, b = 20;
    printf("sum is :%d",a+b);
    return 0;
}
```

Total Number of Tokens :

27

3. Count number of tokens

```
int max(int i);
```

Total number of tokens:

7

int, max, (, int, i,), ;

4. Find the number of tokens in the following C statement

```
printf("i=%d, &i=%x",i,&i);
```

Total number of tokens:

- a. 3
- b. 26
- c. **10**
- d. 20

5. How many tokens are contained in the following FORRTAN statement

```
IF (NUM EQ MAX) GOTO 500
```

Total number of tokens:

- a. 8
- b. 10
- c. 22
- d. **24**

- Consider the following C program

```
int stranger (int x)
{
    if (x<=0) return 0;
    if (x%2!=0) return x-1;
    return 1 + stranger (x-1);
}
```

Total number of tokens:

42

The number of tokens in the following C code segment is

```
switch(inputvalue)
{
    case 1 : b = c * d; break;
    default : b = b++; break;
}
```

```
switch (inputvalue)
{
    case 1 : b = c * d ; break ;
    default : b = c ++ ; break ;
}
```

Total number of tokens:

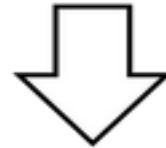
- a. 27
- b. 29
- c. **26**
- d. 24

```
main()
{
    int x = 10 , *P ;
    int y = x ++ ;
    char * q ;
    P = & x ; q = 'A' ;
    if(*P>=10)
    {
        *P = x + 100;
    }
    else
    {
        printf("%d" , x);
        /*comment*/
    }
}
```

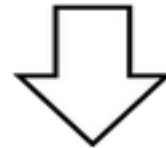
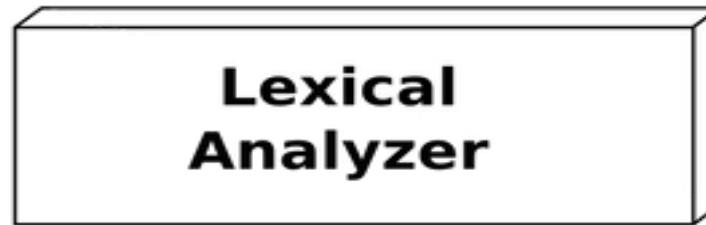
Total number of tokens:

58

i	f	(		x		>		3	.	1	
---	---	---	--	---	--	---	--	---	---	---	--



Character Stream



Token Stream

KEYWORD	BRACKET	IDENTIFIER	OPERATOR	NUMBER
"if"	" ("	"x"	">"	"3.1"

PATTERN

- A **pattern** is a *rule matched by a sequence of characters*(lexemes) used by the token.
- It can be defined by **regular expressions** or **grammar rules**.
- It is the description of class of tokens.
- In the case of a keyword as a token, the pattern is just the sequence of characters that form the keyword.

Token	Sample Lexeme	Informal Description of Pattern
ID	x, y, n0,pi	Letter followed by letters and digits
NUM	-123, 1.456e-5	Any numeric constant
IF	if	if
LPAREN	(	(
LITERAL	``Hello''	Any string of characters (except ``) between `` and ``
RELATION	<,<=,=,< >,>=,>	< or <= or = or < > or >= or >
CONST	Const	Constant

Criteria	Token	Lexeme	Pattern
Definition	Token is basically a sequence of characters that are treated as a unit as it cannot be further broken down.	It is a sequence of characters in the source code that are matched by given predefined language rules for every lexeme to be specified as a valid token.	It specifies a set of rules that a scanner follows to create a token.
Interpretation of type Keyword	All the reserved keywords of that language(main, printf, etc.)	int, goto	The sequence of characters that make the keyword.
Interpretation of type Identifier	Name of a variable, function, etc	main, a	It must start with the alphabet, followed by the alphabet or a digit.

Criteria	Token	Lexeme	Pattern
Interpretation of type Operator	All the operators are considered tokens.	+, =	+, =
Interpretation of type Punctuation	Each kind of punctuation is considered a token. e.g. semicolon, bracket, comma, etc.	(,), {, }	(,), {, }
Interpretation of type Literal	A grammar rule or boolean literal.	"Welcome to Compiler Design!"	any string of characters (except ' ') between " and "

Attributes for Tokens

- The lexical analyzer collects information about tokens into their associated attributes.
- Token** <Token_name, optional_attribute>
- The **tokens** influence **parsing decisions** and **attributes** influence the **translation of tokens**.
- Usually a token has a single attribute i.e. pointer to the symbol table entry in which the information about the token is kept.
- The token names and associated attribute values for the Fortran statement are written below as a sequence of pairs.

E = M * C ** 2

Lexeme	<Token, token_attribute>
E	<id, pointer to symbol-table entry for E>
=	<assign_op>
M	<id, pointer to symbol-table entry for M>
*	<mult_op>
C	<id, pointer to symbol-table entry for C>
**	<exp_op>
2	<number, integer value 2>